

Transformer-Based Text Classification Using BERT

Assignment Report — Dataset Preparation, Training, Optimization, Evaluation and Deployment

1. Introduction

Transformers are deep-learning architectures designed to process sequential data efficiently using attention mechanisms. They have become a foundation of modern Natural Language Processing (NLP) because they can capture relationships between words even when those words are far apart in a sentence.

This project applies a transformer model to a real-world text-classification task. The selected application is binary sentiment classification of movie reviews, where each review is classified as positive or negative.

2. Transformers Explained

- **Tokenization:** Converts text into tokens that can be processed by a model.
- **Embeddings:** Represent tokens as numerical vectors.
- **Self-Attention:** Allows the model to determine which tokens are important in relation to other tokens.
- **Positional Information:** Helps the model represent token order.
- **Encoder/Decoder:** Transformer architectures may use encoder layers, decoder layers, or both depending on the task.
- **Transfer Learning:** A pre-trained model can be adapted to a specific downstream task.

3. Dataset Selection

The IMDb Movie Reviews dataset is selected for this project. It contains movie-review text labeled for sentiment. The dataset is suitable for demonstrating supervised text classification because the input is natural-language text and the target is a binary sentiment label.

- Input: Movie review text
- Output: Positive or Negative sentiment
- Task type: Binary text classification
- Possible source: Kaggle / IMDb dataset mirror
- Preprocessing: Cleaning, tokenization, truncation and padding

4. Environment Setup

The project can be implemented in Python using the Hugging Face Transformers ecosystem together with PyTorch and supporting data-science libraries.

Typical packages: Python, transformers, datasets, torch, evaluate, scikit-learn, pandas and numpy.

Example installation: `pip install transformers datasets torch evaluate scikit-learn pandas numpy`

5. Data Preparation

- Load the dataset into a DataFrame or Hugging Face Dataset.
- Check for missing or duplicated records.
- Inspect class distribution.

- Split data into training, validation and test sets when required.
- Use a BERT tokenizer to convert reviews into token IDs and attention masks.
- Apply truncation and padding to maintain a suitable input length.
- Keep the test set separate so that final performance is measured on unseen data.

6. Model Construction

BERT (Bidirectional Encoder Representations from Transformers) is used as the classification model. A pre-trained BERT model can be loaded with a sequence-classification head and fine-tuned on the sentiment dataset.

The classification head receives the representation learned by BERT and predicts one of the two sentiment classes. During fine-tuning, model parameters are updated using labeled training examples.

7. Training Protocol

- Initialize a pre-trained BERT checkpoint.
- Tokenize the training and validation datasets.
- Select hyperparameters such as learning rate, batch size and number of epochs.
- Train using the training set.
- Evaluate on the validation set after training steps or epochs.
- Save the best-performing checkpoint.
- Use the untouched test set for final evaluation.

8. Evaluation Metrics

Metric	Purpose
Accuracy	Percentage of predictions classified correctly.
Precision	Measures how many predicted positives are actually positive.
Recall	Measures how many actual positives are correctly identified.
F1-score	Harmonic mean of precision and recall.

9. Confusion Matrix

A confusion matrix can be used to inspect true positives, true negatives, false positives and false negatives. It helps identify whether the model performs similarly across the positive and negative classes.

10. Optimization Techniques

- Tune the learning rate and batch size.
- Use an appropriate number of training epochs to reduce underfitting and overfitting.
- Use weight decay or other regularization where appropriate.
- Apply early stopping or select the best validation checkpoint.
- Use a maximum sequence length appropriate for the dataset.
- Consider freezing selected layers for faster experimentation.

- Use learning-rate scheduling and warm-up when appropriate.
- Use mixed-precision training when supported by the hardware.

11. Expected Findings

A fine-tuned transformer is expected to learn contextual language patterns and provide strong performance on movie-review sentiment classification. The final report should include the actual metrics produced by the student's training run rather than fabricated values.

Final Results to Insert: Accuracy: _____ | Precision: _____ | Recall: _____ | F1-score: _____

12. Deployment Options

- **Web API:** Deploy the model behind a FastAPI or Flask endpoint.
- **Web Application:** Create a simple interface where users enter text and receive a sentiment prediction.
- **Cloud Deployment:** Host the inference service on a cloud platform.
- **Containerized Deployment:** Package the application and model using Docker.
- **Model Hub / Inference Service:** Publish the model and expose it through an inference endpoint.
- **Local Deployment:** Run the trained model on a personal computer for offline predictions.

13. GitHub Repository Structure

A suitable repository can contain the following files:

- README.md — project description and setup instructions
- train.py — model training code
- predict.py — inference/prediction code
- requirements.txt — Python dependencies
- notebooks/ — optional experimentation notebook
- results/ — evaluation outputs and confusion matrix
- .gitignore — excludes unnecessary files and secrets

14. GitHub Submission

Repository Link: Paste your GitHub repository URL here before submission.

Important: Never upload API keys, passwords, private datasets, or other secrets to GitHub.

15. Conclusion

This project demonstrates the complete workflow for applying Transformers to a real-world NLP problem. The IMDb sentiment-classification task provides a practical setting for dataset preparation, BERT-based model construction, training, evaluation and optimization. The trained model can subsequently be deployed through an API, web application, cloud service or local inference system.